

Detecting Disaster from Tweets Using Natural Language Processing (NLP)

IE 7500: Milestone 4 - Final Report

Group H: Joshua Davis, Andrew Fox, Mesgana Araia

Abstract

Social media has revolutionized the way we communicate and disseminate critical disaster information. Twitter serves as a platform to gain real-time updates during a natural disaster. However, manually filtering through large volumes of tweets to identify those with critical information can be inefficient and time-consuming. This project uses Natural Language Processing (NLP) techniques to develop a machine learning model capable of distinguishing between disaster and non-disaster tweets.

Through this project we train and test multiple models and evaluate their effectiveness through different metrics. These metrics will give insights into each model and aid in determining which model is best suited for the goal of effective identification of disaster-related tweets. We will conclude with limitations of the selected model and future improvements.

Introduction

The rapid influx of information on social media during a natural disaster has transformed emergency response efforts. Platforms like Twitter serve as channels for real-time updates, enabling users to share evacuation protocols, resource availability, and other critical information. However, manually identifying disaster related tweets can defeat the purpose of real-time updates. Misidentifying or delays in processing this information can hinder emergency responses, leading to inefficient resource distribution and potentially endangering lives.

To address these issues, our project seeks to utilize NLP techniques and models to classify tweets as either disaster-related or non-disaster related. The goal of this project is to develop a model that can efficiently and effectively distinguish between critical and non-critical tweets, ensuring emergency response teams receive the information in a timely manner.

We selected a dataset from Kaggle, “Disasters on social media”, to train and test the models (see Appendix A for full data citation). This dataset contains 13 columns with 2 relevant to the project, text and choose_one. The former is the tweet and the latter identifies the tweet as relevant (disaster-related) or not relevant (non-disaster-related). The dataset was also selected for its robustness, with 10,000 labeled tweets.

After careful data selection the next steps were model selection and evaluation metric selection. Through these subsequent steps we were able to determine the best model for our project goal.

Methodology

Preprocessing is the first step and ensures the data is useful and consistent. The text inputs from the dataset require detailed cleaning as social media language is extremely varied. Proper text cleaning ensures comprehensive training and testing of the models. This process includes removing URLs and other special characters, converting text to lowercase, handling contractions, etc. Next, tokenization is the process of splitting text into words (tokens). Tokenization will help the models identify patterns more efficiently. Another major step in preprocessing is the removal of stopwords (e.g. “and”, “the”). This step improves the models’ performance by only focusing on meaningful tokens.

The next phase was model selection where seven models were trained, tested, and evaluated. These models include Logistic Regression, Support Vector Machine (SVM), Naive Bayes, Random Forest, Bidirectional Encoder Representations from Transformers (BERT),

Simple Dense Neural Networks, and Long-Short Term Memory Neural Networks (LSTMs). Logistic regression is a linear classification model known for its interpretability and simplicity. The model predicts the probability of the tweet being disaster-related using a logistic function. It was selected for its simplicity, low computational cost, and fast training. SVMs find the optimal hyperplane that separates disaster and non-disaster tweets. Its effectiveness in high dimensional spaces and robustness to overfitting made SVM an easy addition. Naive Bayes classifies tweets based on Bayes' Theorem, which assumes words are conditionally independent. This model was selected because of its real-time classification possibilities. Random Forest is an ensemble of decision trees that aggregates predictions from each tree to produce a robust output. This model was selected because of its capability to handle non-linear relationships. BERT is a pre-trained transformer that uses an attention mechanism to understand context. The transformer is pre-trained on a large text dataset providing BERT with a strong foundational understanding of linguistic nuances. It also allows for increased contextual understanding due to its bidirectional nature. BERT was selected because of these strengths. Neural networks were selected because of their effective handling of non-linear relationships, along with their flexibility and deep learning structure. Lastly, LSTM was selected for their handling of variable length text and ability to capture long-range dependencies.

The effectiveness of each model was evaluated using accuracy, precision, recall, and F1-scores. Accuracy measures the overall correctness of the model. This measurement is a great start but can be misleading if the data is imbalanced, meaning there are more non-disaster tweets than disaster-related tweets. Precision, recall, and F1-scores build upon accuracy and provide clearer insights into the model's effectiveness. Precision measures how reliable the model is when it predicts a tweet as a disaster. High precision indicates a low false alarm rate, minimizing wasted emergency resources. Recall measures how well the model captures all actual disaster tweets. High model recall ensures the model does not miss any tweets about a disaster. F1-score is the average of precision and recall, providing an overview of the model's effectiveness. The composite nature of F1-scores make it a crucial metric in the model evaluation and the deciding metric during optimal model selection.

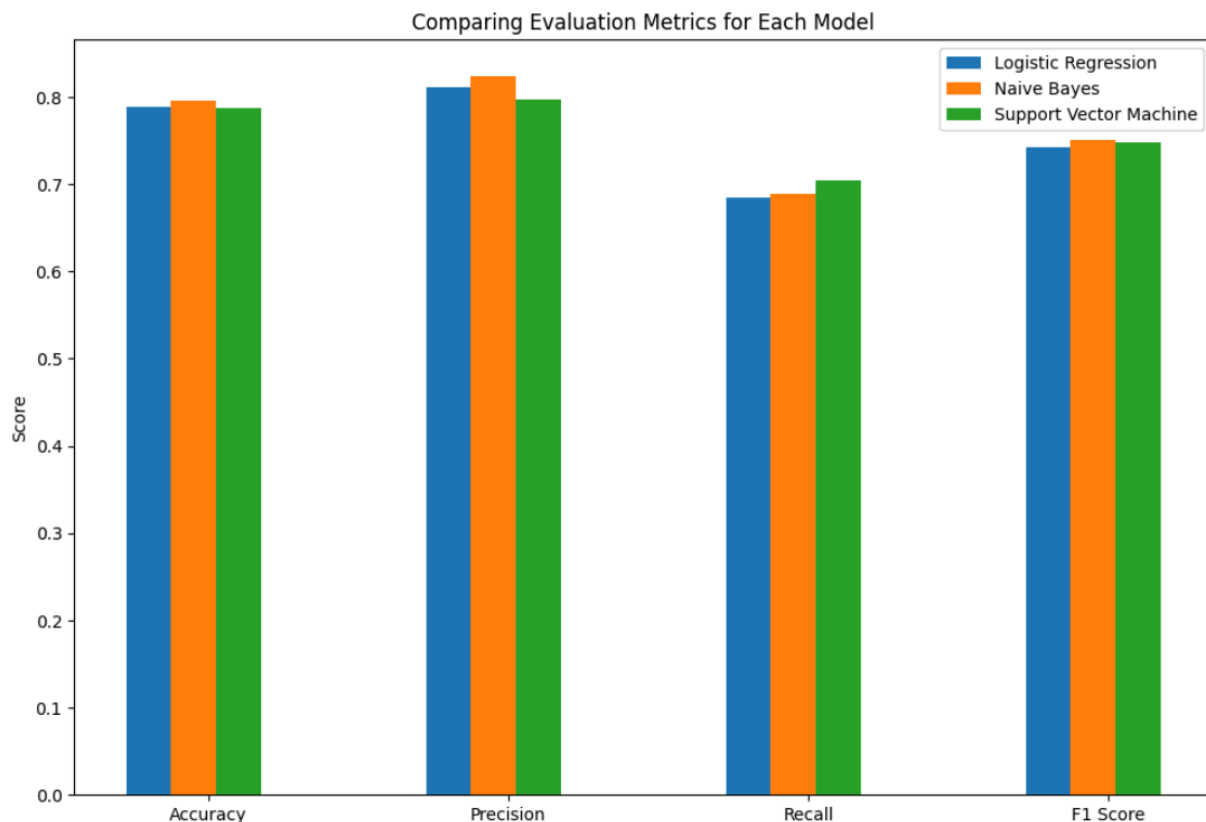
The full approach for each model can be found [here](#).

Results and Discussion

The logistic regression model achieved an accuracy of 79%, with 81% precision, 68% recall, and an F1-score of 0.74. Of note here is the low recall of the logistic regression model, indicating that the model misses a noticeable portion of true disaster-related tweets. This suggests that logistic regression tends to favor fewer false positives at the expenses of increased false negatives.⁷⁵⁰

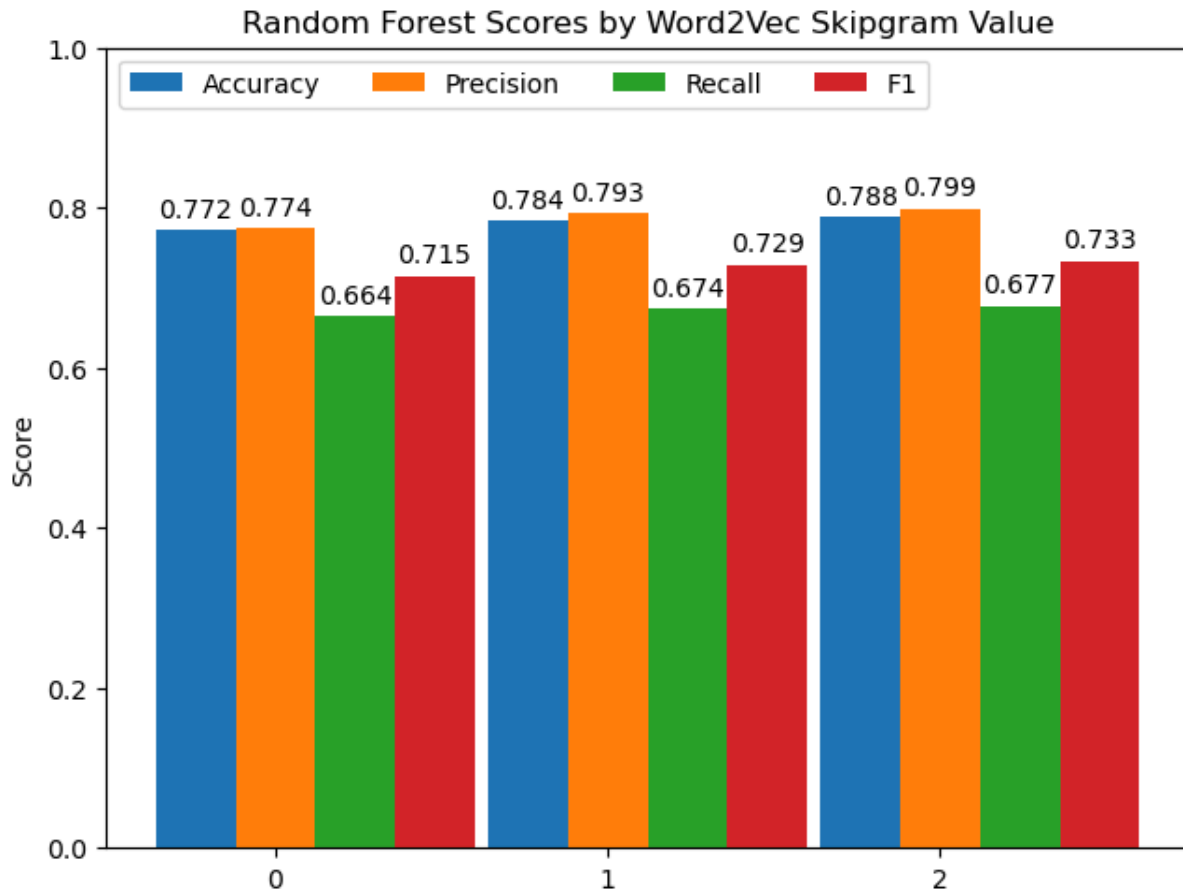
The Naive Bayes classifier achieved a marginally better performance with an accuracy of 80%, with 82% precision, 69% recall, and an F1-score of 0.75. The precision and recall metrics were quite similar, however, indicating a similar predictive pattern. The solid performance here is notable given Naive Bayes' assumption of feature independence, which is rarely true in real NLP applications and scenarios. This could be due to the fabricated nature of the dataset, although the Naive Bayes classifier remains a powerful and effective text classifier.

The SVM achieved an accuracy of 78%, with 80% precision, 70% recall, and an F1-score of 0.75. The SVM also outperformed logistic regression, and demonstrated a more balanced performance. This indicates an effectiveness at capturing patterns without sacrifice in recall. This is likely due to SVM's ability to draw complex decision boundaries. The figure below provides a visual representation of the three models' performance.

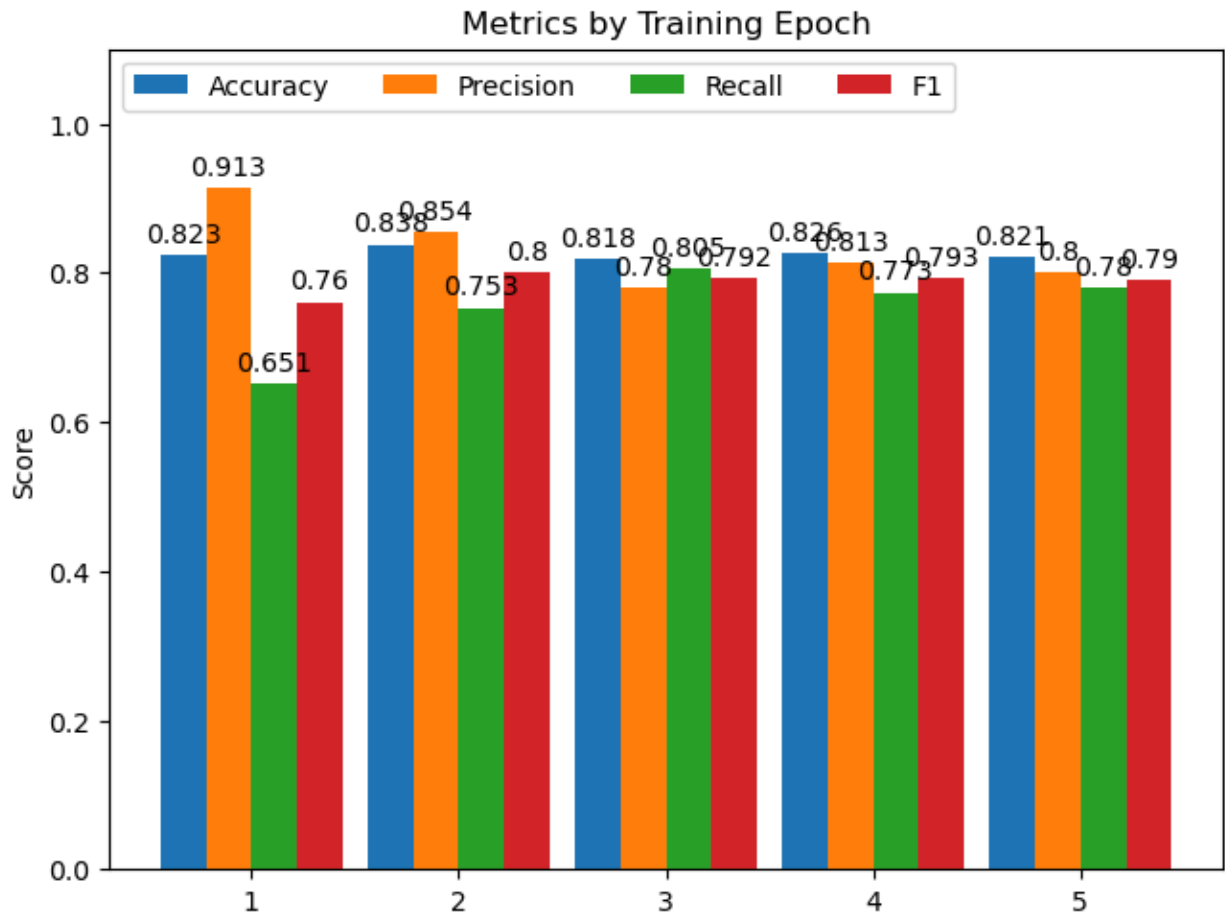


The best result for the Random Forest model based on a Word2Vec conversion achieved an accuracy of 78%, with 79% precision, 67% recall, and an F1-score of 0.72. The model was effective overall, but similar to other models it favored precision over recall. Because random forests are an ensemble of decision trees, they aggregate predictions to reduce

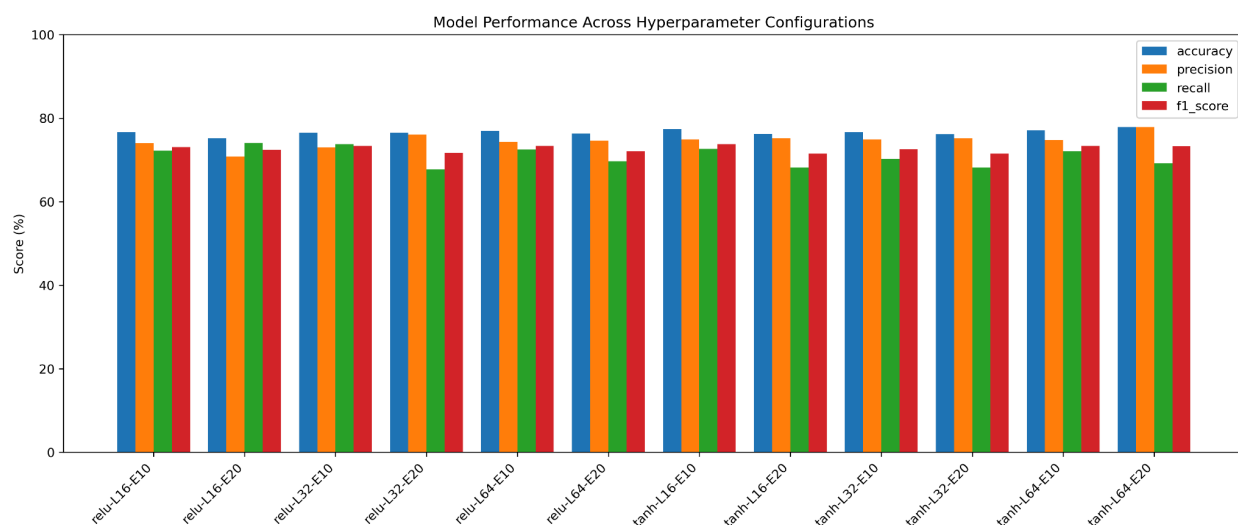
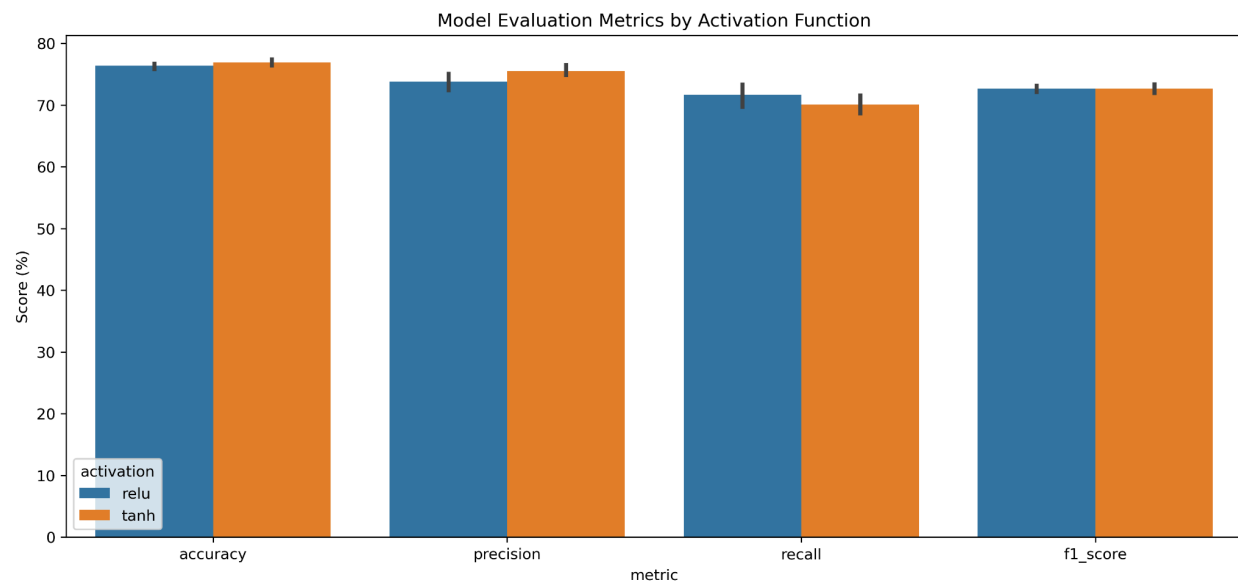
overfitting; the random forest here may have begun to favor a majority class in the dataset. Additionally, random forests are known to struggle with sparse text data. The figure below plots a visualization of the scores across different skipgram configurations for the Word2Vec mapping.



The BERT model achieved an accuracy of 84%, with 85% precision, 75% recall, and an F1-score of 0.80. This was the best performing model, and highlights BERT's strength in contextual and semantic understanding. What most likely contributed to its performance was the bidirectional attention mechanism, which permitted a fuller understanding of the text in both forward and backwards readings of tweets. The model is also pre-trained, which provides a foundational understanding of any linguistic nuances. It is worth noting that this comes with an increased computational cost and training complexity, but BERT prevails as a strong architecture in handling real-world, noisy, and context-dependent text data. Interestingly, the BERT Model achieved its best performance while training on only the 2nd of the 5 epochs used, as shown in the figure below.



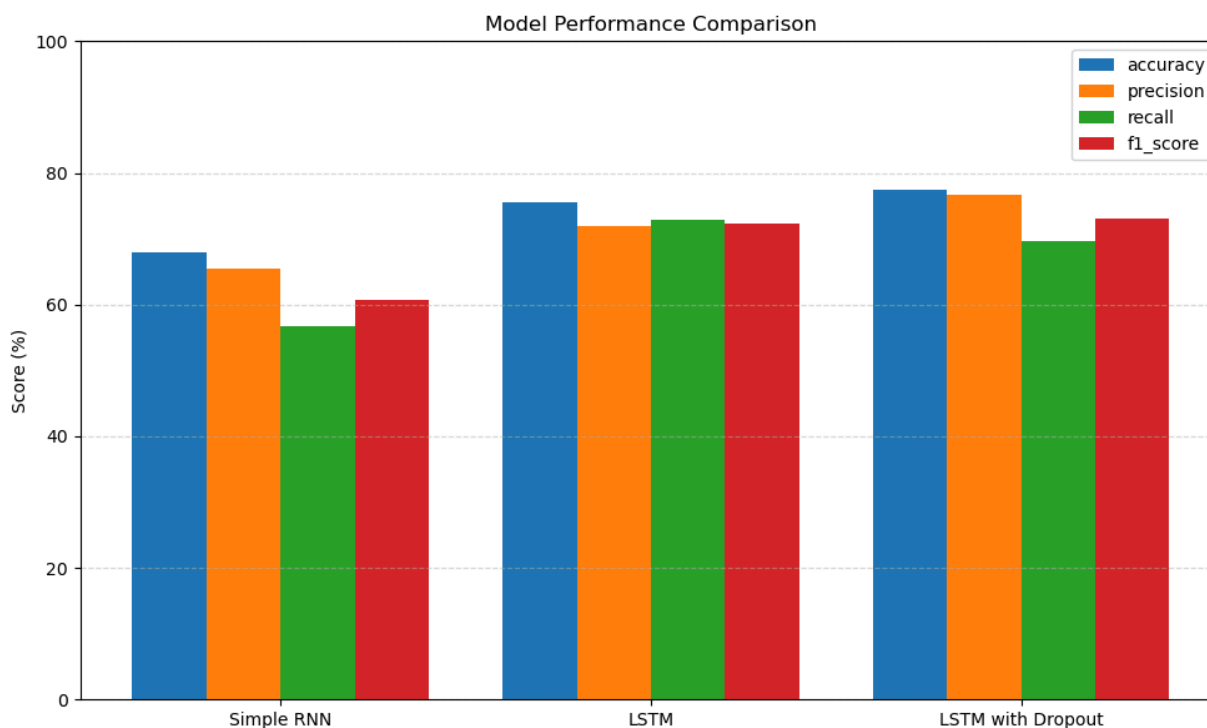
Next we discuss the neural network models, which included a simple dense network, a simple recurrent network, and LSTM models with and without dropout regularization. Hyperparameter tuning was performed to evaluate the performance of multiple simple dense neural networks. The best performing network used ReLu activation, had 16 hidden layers, and ran for 10 epochs. It achieved an accuracy of 76%, with 78% precision, 83% recall, and an F1-score of 0.81. It should be noted, however, that there was no significant difference in performance across parameter configurations. This, along with the general performance of each model, is visualized in the figures below.



There are several reasons why different parameter configurations led to similar results. First, the dataset may be too small or simple, not requiring a deep learning method. The simple model architecture used here may be sufficient to capture the data's essential features, thereby nullifying the effect of variations in parameters. On a similar token, adding more complexity to the model like extra layers and additional epochs may lead to diminishing returns without affecting the model's generalization. Second, the model may be underfitting or overfitting the data. Without regularization, the model may have fit too closely to the data. This could explain both a less than ideal performance and the limited improvements across different parameter configurations. Likewise, with too few epochs, the model may underfit without enough time to optimize itself.

Next, a simple recurrent neural network (RNN) and long short-term memory (LSTM) networks with and without dropout were evaluated. They were trained with the same parameters of the best performing simple dense network, with the exception of 64 hidden units as opposed to 16. The simple RNN achieved an accuracy of 63%, with 70% precision, 65% recall, and an

F1-score of 0.67. The LSTM without dropout achieved an accuracy of 75%, with 79% precision, 79% recall, and an F1-score of 0.79. The LSTM with dropout achieved an accuracy of 76%, with 78% precision, 83% recall, and an F1-score of 0.80. These results are depicted in the figure below.



Of these models, the LSTM with dropout performed best and the performance of the simple RNN was rather dismal. This is likely due to the fact that RNNs struggle with the vanishing gradient problem and often fail to capture contextual long-range dependencies within the text. This highlights the difference between simple RNNs and LSTMs and demonstrates why the latter models performed significantly better than the former. Thanks to the gating mechanisms that preserve long-range dependencies in sequential data, the LSTM model could better understand the contextual nuances and patterns present in the tweets, particularly those that differentiate disaster-related from non-disaster-related tweets.

The LSTM with dropout regularization marginally improved performance from the LSTM without dropout. The regularization helps prevent overfitting, suggesting this model achieved a more robust understanding of the data. What's significant is that the LSTM with dropout performed very similarly to the simple dense neural network. This implies that for this dataset, which is relatively short and simple, the additional temporal modeling complexity of LSTMs provides little improvements in comparison to simple feedforward mechanisms. This supports the idea that the simple neural network is sufficient enough for this problem, and additional complexity may lead to diminishing returns.

Based on the results of all models, we have selected BERT as the model that aligns with our project goal the most. Its strengths in pre-training and contextual understanding are exemplified by the highest F1-score of all tested models.

Conclusion and Future Work

Generally speaking, all models showed a relatively decent ability to accurately predict, which is a solid indication that better performing models can be created which will have improved results. The model results could be improved in several different ways. First, we could consider increasing the complexity of the model at the cost of performance such as by increasing the number of epochs available for training or increasing the complexity of each epoch by increasing batch sizes. Additionally, the model could be improved by performing additional data cleansing and normalization. For example, by identifying and expanding abbreviations or reviewing the most common words in the corpus to add custom stop words. Furthermore, the model output could possibly be improved by considering an ensemble construction considering the output of several models. Several model types, especially the neural networks and LSTM, could also have performance improved if additional labelled data were made available for training, though that will negatively impact the performance of the models. A last consideration is that the data available for model development from this source is relatively well aged by social media standards, and thus may be a poor predictor in the current environment, which would require newer data to resolve.

Appendix A

szelee.(n.d.). Kaggle.*Disasters on social media*. Retrieved January 24, 2025, from <https://www.kaggle.com/datasets/szelee/disasters-on-social-media>